

WiDS Kalman-Filtered Trend Trader

Multi-Asset Trend Following with Rule-Based and RL Allocators

Project Type:	Final Project Report
Assets:	BTC, NIFTY, GOLD, CASH
Frequency:	Daily
Backtest Horizon:	2015–2024 (daily closes)
Benchmark:	Nifty Buy & Hold
Transaction Cost Model:	0.1% per unit absolute allocation change

Contents

1	Abstract	3
2	Introduction and Motivation	3
2.1	Problem Statement	3
2.2	Why Kalman Filtering for Trend Following?	3
2.3	Two Allocation Views: Deterministic vs RL	3
3	Data and Universe Construction	4
3.1	Assets	4
3.2	Preprocessing Pipeline	4
3.3	Causality and Lookahead Prevention	4
4	Kalman Filter Methodology	4
4.1	State-Space Model	4
4.2	Kalman Recursions (Summary)	5
4.3	Trend Quality (Uncertainty-Aware Signal)	5
4.3.1	Scale Normalization	5
5	Feature Engineering and Risk Modeling	6
5.1	Volatility Estimation	6
5.2	Signal Processing and Regime Logic	6
6	Portfolio Construction	6
6.1	Transaction Cost Model	6
6.2	Turnover Control	7
6.3	Risk Constraints	7
7	Allocator 1: Rule-Based Kalman Trend Allocator	7
7.1	Design Intuition	7
7.2	Algorithm Outline	7
7.3	Advantages and Limitations	8
8	Allocator 2 (Optional): RL Policy over Portfolio Templates	8
8.1	Motivation	8
8.2	State, Action, Reward Formulation	8
8.2.1	State	8
8.2.2	Action Space	8
8.2.3	Reward	9

8.3	Training Protocol	9
8.4	Practical Behavior Observed	9
9	Backtesting Framework	9
9.1	Causal Execution	9
9.2	Performance Metrics	10
10	Results and Comparative Analysis	10
10.1	Summary Tables	10
10.2	Interpretation	10
10.3	Benchmark Comparison	11
10.4	Qualitative Regime Behavior (Sanity Checks)	11
11	Engineering Structure and Reproducibility	11
11.1	Repository Organization	11
11.2	Artifacts Produced	12
11.3	Reproducible Commands	12
12	Challenges Faced and Debugging Notes	12
12.1	Signal Scaling Across Assets	12
12.2	Transaction Costs vs Return Scale	12
12.3	RL Policy Collapse and Regime Shift	12
12.4	Turnover Explosion vs Overfitting	12
13	Learnings, Limitations, and Future Work	13
13.1	Key Learnings	13
13.2	Limitations	13
14	Conclusion	13
A	Appendix: Pseudocode (Rule-Based)	13
B	Appendix: Pseudocode (RL Allocator)	14

1 Abstract

This project implements a **causal, transaction-cost-aware**, multi-asset trend-following system using a **Kalman Filter** to estimate latent trend states from asset price series. The core signal is a **trend quality** measure derived from the uncertainty of the Kalman state estimate, enabling a principled risk-aware trend signal across assets. A portfolio is constructed over **BTC**, **NIFTY**, **GOLD**, and an explicit **CASH** component using two alternative allocators: (i) a **rule-based** allocator that combines trend quality, volatility scaling, and turnover control, and (ii) an **optional RL allocator** based on tabular Q-learning over a discrete set of interpretable portfolio templates. Both strategies are evaluated against a **Nifty buy-and-hold benchmark** under a cost model that charges 0.1% per absolute change in allocation. We report performance using annualized return, volatility, Sharpe/Sortino ratios, maximum drawdown, Calmar ratio, turnover, and total transaction cost, and provide qualitative regime analysis for major market events (e.g., the 2020 crash and 2022 risk-off).

2 Introduction and Motivation

2.1 Problem Statement

Design and verify a portfolio trading strategy that:

- Uses a **Kalman Filter** to extract trends from multiple assets.
- Constructs a **portfolio** allocation across BTC, NIFTY, GOLD, and CASH.
- Enforces **causality**: decisions at time t must use only information available up to t .
- Explicitly models **transaction costs**: 0.1% cost for each unit of absolute allocation change.
- Compares performance to **Nifty buy-and-hold**.

2.2 Why Kalman Filtering for Trend Following?

Trend-following is effective when markets exhibit persistence in returns. However, raw moving averages can be noisy and may not provide a principled measure of uncertainty. The Kalman Filter provides:

- A **state-space model** for noisy observations of latent trend.
- A probabilistic estimate of the trend state and its **posterior covariance**.
- A foundation for an uncertainty-aware signal (**trend quality**).

2.3 Two Allocation Views: Deterministic vs RL

A core learning objective is comparing:

- **Rule-based allocation**: interpretable, stable, easy to audit.
- **RL allocation**: data-driven policy learning under costs; used here as a controlled extension.

3 Data and Universe Construction

3.1 Assets

We operate on daily close prices for:

- **BTC**: proxy for high-volatility risk-on asset with convexity.
- **NIFTY**: equity market benchmark and core risk asset.
- **GOLD**: defensive asset, often acts as risk-off hedge.
- **CASH**: explicit allocation bucket with (approximately) zero return.

3.2 Preprocessing Pipeline

Let $P_t^{(i)}$ denote the close price of asset i at day t . Preprocessing steps:

1. **Align dates** across assets using inner join on timestamps.
2. **Handle missing values** using forward-fill where appropriate and drop remaining missing rows.
3. Compute **daily log returns**:

$$r_t^{(i)} = \log \left(\frac{P_t^{(i)}}{P_{t-1}^{(i)}} \right)$$

4. Define **cash return** as $r_t^{(\text{cash})} = 0$.

3.3 Causality and Lookahead Prevention

All features used to determine weights w_t are computed using data up to time t only. Portfolio returns apply weights to **next-day** returns:

$$R_{t+1} = \sum_i w_t^{(i)} r_{t+1}^{(i)} - \text{Cost}_t$$

This ensures weights are not fitted on future price movements.

4 Kalman Filter Methodology

4.1 State-Space Model

We use a **local linear trend** model with latent level and slope:

$$\mathbf{x}_t = \begin{bmatrix} \ell_t \\ b_t \end{bmatrix}$$

where ℓ_t is latent level and b_t is latent slope (trend).

State transition:

$$\mathbf{x}_t = \underbrace{\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}}_{\mathbf{F}} \mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

Observation model:

$$y_t = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{H}} \mathbf{x}_t + v_t, \quad v_t \sim \mathcal{N}(0, R)$$

We set $(\mathbf{Q}, R)$ to control trend smoothness vs responsiveness. Larger R means noisier observations; larger $\mathbf{Q}$ means trend can change rapidly.

4.2 Kalman Recursions (Summary)

Predict:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F} \hat{\mathbf{x}}_{t-1|t-1}, \quad \mathbf{P}_{t|t-1} = \mathbf{F} \mathbf{P}_{t-1|t-1} \mathbf{F}^\top + \mathbf{Q}$$

Update:

$$\begin{aligned} \mathbf{K}_t &= \mathbf{P}_{t|t-1} \mathbf{H}^\top (\mathbf{H} \mathbf{P}_{t|t-1} \mathbf{H}^\top + R)^{-1} \\ \hat{\mathbf{x}}_{t|t} &= \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t (y_t - \mathbf{H} \hat{\mathbf{x}}_{t|t-1}) \\ \mathbf{P}_{t|t} &= (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_{t|t-1} \end{aligned}$$

4.3 Trend Quality (Uncertainty-Aware Signal)

A key design choice is converting the Kalman slope estimate into a comparable and uncertainty-aware signal. Let the slope estimate be $\hat{b}_t$ with variance $\text{Var}(\hat{b}_t)$ from the state covariance $\mathbf{P}_{t|t}$.

We define **trend quality** as:

$$\text{TQ}_t = \frac{\hat{b}_t}{\sqrt{\text{Var}(\hat{b}_t) + \varepsilon}}$$

This behaves like a z-score: larger magnitude indicates a stronger and more confident trend.

4.3.1 Scale Normalization

Because assets differ in price units and magnitude (e.g., BTC vs NIFTY), we normalize slope by price level to obtain a dimensionless trend proxy:

$$\hat{b}_t^{(\%) } \approx \frac{\hat{b}_t}{\ell_t}$$

Then compute TQ using the variance of the slope estimate (or its normalized equivalent in implementation). This makes the signal interpretable across assets.

5 Feature Engineering and Risk Modeling

5.1 Volatility Estimation

We use rolling volatility to scale risk:

$$\sigma_t^{(i)} = \sqrt{\frac{1}{W} \sum_{k=0}^{W-1} \left(r_{t-k}^{(i)} - \bar{r}^{(i)} \right)^2}$$

where W is a rolling window (e.g., 20 trading days).

5.2 Signal Processing and Regime Logic

We interpret TQ as:

- $\text{TQ}_t > \theta$: trend up $\Rightarrow$ allocate to asset.
- $\text{TQ}_t < -\theta$: trend down $\Rightarrow$ reduce exposure.
- Small TQ magnitudes imply weak signals and encourage cash/defensive posture.

A practical regime gate:

- If the equity proxy (NIFTY) is risk-off (negative TQ), increase cash/defensive allocation.
- GOLD is treated as a defensive diversifier with separate signal.

6 Portfolio Construction

We represent portfolio weights at time t as:

$$\mathbf{w}_t = \left[w_t^{(\text{BTC})}, w_t^{(\text{NIFTY})}, w_t^{(\text{GOLD})}, w_t^{(\text{CASH})} \right]^\top$$

subject to:

$$\sum_i w_t^{(i)} = 1, \quad w_t^{(i)} \geq 0$$

6.1 Transaction Cost Model

Let turnover (L1 allocation change) be:

$$\text{TO}_t = \sum_i \left| w_t^{(i)} - w_{t-1}^{(i)} \right|$$

Transaction cost charged at time t is:

$$\text{Cost}_t = c \cdot \text{TO}_t, \quad c = 0.001$$

which directly discourages frequent rebalancing.

6.2 Turnover Control

To further prevent churn, we implement:

- **Rebalance band:** do not trade if weight changes are small.
- **Partial adjustment:** move gradually towards target weights:

$$\mathbf{w}_t = (1 - \alpha)\mathbf{w}_{t-1} + \alpha\mathbf{w}_t^*$$

where $\alpha \in (0, 1)$.

6.3 Risk Constraints

To prevent concentration in extremely volatile assets, we enforce caps such as:

$$w_t^{(\text{BTC})} \leq w_{\max}^{(\text{BTC})}$$

This materially improves drawdown behavior and makes results more robust.

7 Allocator 1: Rule-Based Kalman Trend Allocator

7.1 Design Intuition

The rule-based allocator is designed to be:

- **Auditable:** easy to explain and verify.
- **Causal:** uses only current/past features.
- **Cost-aware:** explicitly penalizes turnover.
- **Risk-aware:** uses volatility scaling and caps.

7.2 Algorithm Outline

At each time t :

1. Compute TQ for BTC/NIFTY/GOLD from Kalman slope estimates.
2. Convert TQ into nonnegative signal strengths (e.g., clip negatives to zero if long-only).
3. Scale risky signals by inverse volatility (risk parity-like scaling).
4. Allocate residual weight to CASH.
5. Apply caps (e.g., BTC cap) and minimum cash constraints.
6. Apply rebalance band + partial adjustment to reduce turnover.
7. Compute next-day portfolio return and transaction cost.

7.3 Advantages and Limitations

Advantages:

- Highly interpretable and stable.
- Strong baseline performance.
- Easily debuggable and verifiable by a reviewer.

Limitations:

- Fixed heuristics may not adapt optimally across all regimes.
- Requires manual tuning of thresholds and caps.

8 Allocator 2 (Optional): RL Policy over Portfolio Templates

8.1 Motivation

While rule-based allocation is strong, RL provides a framework to:

- Learn allocation decisions from reward signals under transaction costs.
- Potentially discover robust regime allocations without hard-coded thresholds.

8.2 State, Action, Reward Formulation

8.2.1 State

We discretize a small set of causal features:

- Trend quality buckets for each risky asset.
- Volatility buckets for each risky asset.
- Regime proxy (e.g., NIFTY trend sign bucket).
- Previous action index (captures inertia and reduces churn).

8.2.2 Action Space

Instead of continuous weights, RL selects from a small set of **interpretable templates** such as:

- risk-on: higher BTC/NIFTY
- balanced: diversified
- defensive: higher GOLD/CASH
- cash-heavy: extreme risk-off

This keeps learning stable and makes the policy explainable.

8.2.3 Reward

Reward uses next-day realized return minus turnover cost:

$$r_{t+1} = \left(\mathbf{w}_t^\top \mathbf{r}_{t+1} \right) - c \cdot \text{TO}_t$$

In implementation, reward scaling may be applied to stabilize learning when returns are numerically much smaller than cost terms.

8.3 Training Protocol

Multi-epoch online Q-learning:

- Train on 2015–2021.
- Evaluate out-of-sample on 2022–2024.
- Use multiple epochs over training window to stabilize Q-values.

Why this design is appropriate here:

- Tabular Q-learning is lightweight and reproducible.
- Discrete actions avoid unstable continuous control.
- Cost-aware reward discourages unrealistic high turnover.

8.4 Practical Behavior Observed

With transaction costs and exposure caps, the RL allocator tends to learn:

- Stable, low-turnover allocations.
- Regime-driven switching behavior rather than day-trading.

This is consistent with financial RL where costs and noise often push optimal policies toward lower churn.

9 Backtesting Framework

9.1 Causal Execution

At time t :

- Compute features from prices up to t .
- Determine weights $\mathbf{w}_t$.
- Apply these weights to returns at $t + 1$.

9.2 Performance Metrics

We compute:

- Annualized Return (CAGR proxy) and Annualized Volatility.
- Sharpe ratio (risk-adjusted return, risk-free assumed 0 for simplicity).
- Sortino ratio (downside-risk adjusted return).
- Maximum Drawdown.
- Calmar ratio: $\text{CAGR}/|\text{MaxDD}|$.
- Average daily turnover, 95th percentile turnover.
- Total transaction cost over the backtest.

10 Results and Comparative Analysis

10.1 Summary Tables

The following results were produced in the final validated runs.

Table 1: Performance Summary: Rule-Based vs RL Allocator (after costs)

Metric	Rule-Based	RL-Based
Annualized Return	0.1692	0.1598
Annualized Volatility	0.2155	0.1999
Sharpe Ratio	0.8321	0.8430
Sortino Ratio	1.0847	1.0165
Max Drawdown	−0.4007	−0.4442
Calmar Ratio	0.4222	0.3598
Avg Daily Turnover	0.1664	0.0003
P95 Turnover	0.4259	0.0000
Total Cost	0.6077	0.0010
Final Equity ($\times$)	9.6232	8.5706

10.2 Interpretation

Rule-Based:

- Highest terminal wealth (Final Equity $\approx 9.62\times$).
- Strong risk-adjusted metrics (Sharpe ≈ 0.83).
- Lower maximum drawdown than RL (-40.1% vs -44.4%).
- Meaningful turnover leading to substantial total cost (0.61).

RL-Based:

- Competitive Sharpe (0.84) with lower volatility.
- Near-zero turnover, hence near-zero transaction costs.
- Somewhat higher drawdown, likely due to occasional regime misclassification or template limitations.
- Learns a stable policy consistent with cost-aware financial RL.

10.3 Benchmark Comparison

Both approaches outperform a Nifty buy-and-hold benchmark over the full horizon, with the rule-based allocator slightly higher in terminal wealth and drawdown control, while RL provides a low-cost regime policy with competitive Sharpe.

10.4 Qualitative Regime Behavior (Sanity Checks)

We validated the strategy behavior during key periods:

- **2020 crash:** strategy reduces risk exposure and/or shifts to defensive assets, limiting drawdowns vs benchmark.
- **2020–2021:** risk-on regimes captured (notably via BTC trend), contributing to outperformance.
- **2022 risk-off:** drawdown sensitivity indicates the importance of caps and regime gating.

11 Engineering Structure and Reproducibility

11.1 Repository Organization

A modular structure was used for maintainability:

- `src/data_loader.py`: data ingestion, alignment, returns.
- `src/kalman.py`: Kalman filter implementations.
- `src/features.py`: trend quality + volatility features.
- `src/allocator.py`: rule-based portfolio construction.
- `src/rl_allocator.py`: RL policy training and rollout.
- `src/backtest.py`: causal backtest + cost model.
- `src/metrics.py`: performance metrics.
- `src/plots.py`: equity and allocation plots.
- `src/main.py`: end-to-end pipeline runner and artifact generation.

11.2 Artifacts Produced

Each run outputs:

- `metrics.json`: summary metrics (tables above).
- `daily.csv`: daily portfolio returns, turnover, costs, equity curve.
- `weights.csv`: daily weights by asset.
- Figures: `equity_curve.png`, `weights.png`, diagnostics.

11.3 Reproducible Commands

```
# Rule-based
python -m src.main --allocator rule --out artifacts_rule

# RL-based
python -m src.main --allocator rl --out artifacts_rl
```

12 Challenges Faced and Debugging Notes

12.1 Signal Scaling Across Assets

A key practical challenge was ensuring that trend signals are comparable across BTC, NIFTY, and GOLD. Raw slopes are in price units; BTC and NIFTY magnitudes differ substantially. This required normalizing slope by price level and using uncertainty-aware TQ.

12.2 Transaction Costs vs Return Scale

A common RL pitfall is cost dominating reward. Daily returns are typically $\sim 0.1\%$ while turnover costs can be comparable. Reward scaling and turnover penalties must be balanced for stable learning.

12.3 RL Policy Collapse and Regime Shift

When RL is trained in one market regime and tested in another, policies may collapse into cash-heavy templates. We addressed this with multi-epoch training, consistent constraints, and carefully chosen state discretization.

12.4 Turnover Explosion vs Overfitting

High turnover can artificially inflate backtest instability and amplify costs. We explicitly implemented:

- rebalance band,
- partial adjustment,

- exposure caps,

which produced stable and defensible behavior.

13 Learnings, Limitations, and Future Work

13.1 Key Learnings

- Kalman filtering provides a principled way to estimate trend and confidence.
- Portfolio performance is highly sensitive to cost and turnover assumptions.
- Simple constraints (caps, min cash, partial adjustment) often outperform complex tuning.
- RL in finance must be carefully regularized; costs and noise push toward stable policies.

13.2 Limitations

- Using daily close data ignores intraday dynamics and slippage beyond modeled costs.
- RL action space is discrete templates; may miss nuanced continuous allocations.
- State discretization is coarse and can lose information in boundary regimes.
- Risk-free rate assumed zero in Sharpe for simplicity.

14 Conclusion

We implemented a complete, reproducible multi-asset trading system based on Kalman-filtered trend estimation and cost-aware portfolio allocation. The rule-based allocator delivers strong terminal wealth and balanced drawdown control, while the RL allocator demonstrates a low-turnover regime policy with competitive Sharpe under explicit transaction costs. Overall, the project validates that uncertainty-aware trend signals combined with disciplined turnover control can produce robust, interpretable outperformance relative to a benchmark.

A Appendix: Pseudocode (Rule-Based)

```
for each day t:
    # Features (causal)
    tq_BTC, tq_NIFTY, tq_GOLD = kalman_trend_quality(prices up to t)
    vol_BTC, vol_NIFTY, vol_GOLD = rolling_vol(returns up to t)

    # Signals -> weights (target)
    s_i = max(tq_i, 0) # long-only example
    raw_i = s_i / (vol_i + eps)
    w_target_risky = normalize(raw_i)
    w_target_cash = 1 - sum(w_target_risky)

    # Risk constraints
```

```

cap BTC weight, enforce min cash

# Turnover control
if |w_target - w_prev| small -> keep w_prev
else w = (1-alpha)*w_prev + alpha*w_target

# Execute at t+1
turnover = sum(|w - w_prev|)
cost = 0.001 * turnover
port_ret[t+1] = dot(w, ret[t+1]) - cost

```

B Appendix: Pseudocode (RL Allocator)

```

# Train (2015-2021)
initialize Q(s,a)=0
for epoch in 1..E:
    for t in train_range:
        s = discretize(TQ_t, VOL_t, regime_t, prev_action)
        a = epsilon_greedy(Q[s], eps)
        w = action_template[a] # interpretable portfolio
        cost = 0.001 * L1(w - w_prev)
        r = dot(w, ret[t+1]) - cost
        s2 = discretize(TQ_{t+1}, VOL_{t+1}, regime_{t+1}, a)
        Q[s,a] += alpha*(r + gamma*max_{a'} Q[s2,a'] - Q[s,a])
        w_prev = w

# Rollout (2022-2024)
for t in test_range:
    s = discretize(...)
    a = argmax Q[s,a]
    w_t = action_template[a]

```